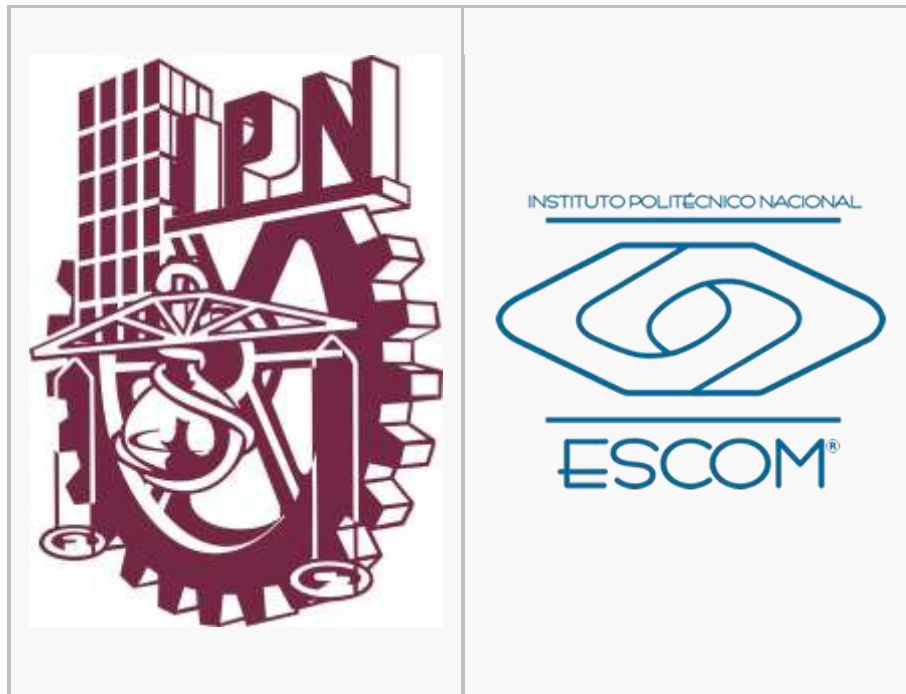




Instituto Politécnico Nacional
Escuela Superior de Cómputo

REPORTE TÉCNICO

Sistema de Estacionamiento Inteligente IoT mediante MQTT

Proyecto con ESP32, sensor ultrasónico HC-SR04, Mosquitto y Node-RED

Nombre del alumno	Marcos Morán Vaquero
Asignatura	Embedded System
Profesor(a)	Miguel Angel Aleman Arce
Grupo	6CM3
Repositorio	estacionamiento-IoT-ESP32

Fecha de entrega	20 de junio de 2026
-------------------------	---------------------

0. Índice

1. Resumen ejecutivo
2. Introducción
3. Planteamiento del problema
4. Objetivos
5. Marco teórico
6. Descripción general del sistema
7. Arquitectura y topología
8. Materiales y componentes
9. Conexiones del circuito
10. Funcionamiento e implementación
11. Comunicación MQTT y dashboard
12. Plan de pruebas y validación
13. Resultados
14. Conclusiones
15. Recomendaciones y trabajo futuro
16. Referencias

1. Resumen ejecutivo

El presente reporte documenta el diseño e implementación de un sistema de estacionamiento inteligente basado en Internet de las Cosas. El prototipo utiliza una ESP32 como nodo embebido principal, un sensor ultrasónico HC-SR04 para estimar la presencia de un vehículo, actuadores físicos para indicar el estado del cajón y comunicación MQTT para publicar información en tiempo real hacia un dashboard en Node-RED. La solución se apoya en Mosquitto como broker local y en una red WiFi compartida entre la ESP32 y la computadora de monitoreo.

El funcionamiento general se basa en la lectura continua de distancia: si el valor medido es menor a 10 cm, el cajón se considera ocupado; si es igual o mayor a 10 cm, se considera libre. Con esta lógica se controlan LEDs, buzzer y servo, además de publicarse los valores de distancia y estado en tópicos MQTT jerárquicos. La documentación se elaboró con base en el repositorio del proyecto y se complementó con criterios técnicos de diseño IoT, comunicación publish/subscribe, seguridad básica y recomendaciones de mejora.

Resultado esperado del prototipo

Un cajón de estacionamiento capaz de indicar localmente su estado mediante actuadores y de reportar en tiempo real distancia y disponibilidad hacia una interfaz web de monitoreo.

2. Introducción

La integración de sensores, microcontroladores y protocolos ligeros de comunicación permite desarrollar soluciones IoT de bajo costo para problemas cotidianos. En el caso de un estacionamiento, conocer la disponibilidad de un espacio en tiempo real reduce incertidumbre, facilita el monitoreo y puede servir como base para sistemas más amplios de administración vehicular.

El proyecto analizado implementa un cajón de estacionamiento inteligente en el que una ESP32 recopila la medición de un sensor ultrasónico, procesa una regla de decisión y publica los datos a través de MQTT. La visualización se realiza mediante Node-RED, herramienta que permite construir flujos y dashboards de forma visual. Además de la interfaz web, el prototipo incluye señalización física por medio de LED verde, LED rojo, buzzer y un servomotor SG90 que simula una barrera automática.

El reporte organiza la información del proyecto en un formato académico y técnico, integrando objetivos, componentes, arquitectura, conexiones, funcionamiento, pruebas, resultados, conclusiones y recomendaciones. Su propósito es servir como documento entregable y como guía replicable para instalar, probar y mejorar el sistema.

3. Planteamiento del problema

En estacionamientos convencionales, el usuario suele identificar visualmente si un espacio está libre u ocupado. Este proceso no escala adecuadamente cuando existen varios cajones o cuando se requiere supervisión remota. La falta de monitoreo automatizado provoca pérdida de tiempo, recorridos innecesarios y ausencia de datos históricos para analizar disponibilidad.

El problema técnico consiste en detectar de manera confiable la presencia de un vehículo en un cajón, transmitir el estado del espacio a una interfaz de monitoreo y accionar indicadores físicos que permitan conocer la condición del cajón sin depender únicamente de la observación directa.

4. Objetivos

4.1 Objetivo general

Diseñar e implementar una arquitectura IoT basada en MQTT que permita monitorear en tiempo real el estado de un cajón de estacionamiento y accionar dispositivos físicos de acuerdo con los datos obtenidos por un sensor ultrasónico.

4.2 Objetivos específicos

- Implementar un nodo publicador MQTT utilizando una ESP32.
- Medir distancia en tiempo real mediante un sensor ultrasónico HC-SR04.
- Configurar Mosquitto como broker MQTT local para gestionar la comunicación.
- Crear un dashboard de monitoreo con Node-RED para visualizar estado y distancia.
- Definir tópicos MQTT jerárquicos, descriptivos y fáciles de mantener.
- Controlar actuadores físicos: LED verde, LED rojo, buzzer y servomotor SG90.
- Validar la comunicación entre sensor, ESP32, broker, dashboard y actuadores.
- Documentar el sistema para que pueda ser replicado y mejorado posteriormente.

5. Marco teórico

5.1 Internet de las Cosas

Internet de las Cosas, o IoT, describe la conexión de dispositivos físicos capaces de medir, procesar y comunicar datos. En este proyecto, la ESP32 representa el nodo IoT encargado de convertir una medición física de distancia en información digital útil para la toma de decisiones.

5.2 ESP32 como nodo embebido

La ESP32 es adecuada para prototipos IoT porque integra conectividad WiFi y capacidad de controlar entradas y salidas digitales. Al programarse desde Arduino IDE con el soporte Arduino Core para ESP32, puede leer sensores, ejecutar lógica local y comunicarse con servicios de red mediante librerías como PubSubClient.

5.3 Sensor ultrasónico HC-SR04

El HC-SR04 permite estimar la distancia a un objeto a partir del tiempo de retorno de una señal ultrasónica. En el prototipo, esta medición se usa como criterio para clasificar el cajón como libre u ocupado. Debido a que el pin ECHO del sensor puede entregar 5 V y la ESP32 trabaja con lógica de 3.3 V, se recomienda emplear un divisor de voltaje para proteger el GPIO de entrada.

5.4 MQTT, broker y modelo publish/subscribe

MQTT es un protocolo de mensajería ligero con modelo publish/subscribe, diseñado para dispositivos con recursos limitados o conexiones donde conviene reducir ancho de banda. En lugar de que cada dispositivo se comunique directamente con otro, los clientes publican mensajes en tópicos y el broker se encarga de distribuirlos a los clientes suscritos.

En este sistema, la ESP32 publica la distancia y el estado del cajón; Mosquitto recibe esos mensajes como broker local; y Node-RED se suscribe a los tópicos para mostrarlos en un dashboard web.

5.5 Node-RED y visualización

Node-RED permite construir flujos de procesamiento conectando nodos visuales. Para el caso del estacionamiento, se emplean nodos MQTT de entrada, elementos de texto, gauge y chart para mostrar estado, medición numérica e historial de distancia.

6. Descripción general del sistema

El sistema está compuesto por cuatro bloques principales: adquisición, procesamiento, comunicación y visualización. La adquisición corresponde al sensor HC-SR04; el procesamiento se realiza en la ESP32; la comunicación se basa en MQTT con Mosquitto; y la visualización se desarrolla en Node-RED.

La ESP32 ejecuta de forma periódica la medición de distancia, compara el valor contra un umbral de 10 cm y determina el estado del cajón. Si el valor es menor al umbral, el sistema reporta OCUPADO, enciende el LED rojo, activa el buzzer con mayor frecuencia conforme el vehículo se

acerca y cierra la barrera simulada con el servo. Si el valor es igual o mayor al umbral, reporta LIBRE, enciende el LED verde, apaga el buzzer y abre la barrera.

Bloque	Elemento principal	Función
Adquisición	Sensor HC-SR04	Medir la distancia entre el sensor y el objeto/vehículo.
Procesamiento	ESP32	Aplicar la lógica de decisión y controlar actuadores.
Comunicación	MQTT + Mosquitto	Publicar y distribuir datos en tópicos jerárquicos.
Visualización	Node-RED Dashboard	Mostrar estado, distancia y comportamiento histórico.
Actuación	LEDs, buzzer y servo	Indicar localmente el estado del cajón y simular barrera.

7. Arquitectura y topología

7.1 Arquitectura funcional

La arquitectura sigue el modelo Publisher-Broker-Subscriber. La ESP32 funciona como publicador MQTT y como controlador de actuadores. Mosquitto opera como broker y Node-RED como suscriptor encargado de la visualización.



Figura 1. Arquitectura funcional del sistema IoT.

7.2 Topología de red

La ESP32 y la computadora donde se ejecutan Mosquitto y Node-RED deben conectarse a la misma red local. La IP de la computadora se configura en el firmware para que la ESP32 pueda conectarse al broker MQTT.

Topología de red local



Condición de operación: la ESP32 y la PC del broker deben estar en la misma red local.

Figura 2. Topología de red local para comunicación entre ESP32, broker y dashboard.

8. Materiales y componentes

8.1 Hardware

Componente	Cantidad	Función dentro del sistema
ESP32 Dev Module	1	Nodo principal IoT, publicador MQTT y controlador de actuadores.
Sensor HC-SR04	1	Medición de distancia para detectar presencia de vehículo.
Servo SG90	1	Simulación de barrera automática de acceso.
LED verde	1	Indicador visual de cajón libre.
LED rojo	1	Indicador visual de cajón ocupado.
Buzzer activo	1	Alerta sonora proporcional a la cercanía del vehículo.
Resistencias de 220 ohmios	2 o más	Protección de LEDs y apoyo para divisor de voltaje.
Protoboard y jumpers	Según montaje	Interconexión de sensor, actuadores y placa ESP32.

8.2 Software

Software / Librería	Uso
Arduino IDE	Programación y carga del firmware en la ESP32.
Arduino Core para ESP32	Soporte de compilación y carga para placas ESP32.
PubSubClient	Cliente MQTT para publicar datos desde la ESP32.
ESP32Servo	Control del servomotor SG90.
Mosquitto	Broker MQTT local.

Node.js	Entorno requerido para ejecutar Node-RED.
Node-RED	Diseño de flujo MQTT y visualización de datos.
node-red-dashboard	Interfaz gráfica web para mostrar estado, distancia e historial.

9. Conexiones del circuito

9.1 Pines utilizados

Elemento	Pin en ESP32	Observación
TRIG HC-SR04	GPIO 5	Salida digital de disparo del sensor ultrasónico.
ECHO HC-SR04	GPIO 18	Entrada mediante divisor de voltaje por señal de 5 V.
LED verde	GPIO 26	Encendido cuando el cajón está libre.
LED rojo	GPIO 27	Encendido cuando el cajón está ocupado.
Buzzer activo	GPIO 25	Alerta sonora intermitente.
Servo SG90	GPIO 14	Señal PWM para apertura/cierre de barrera.

9.2 Conexión del sensor HC-SR04

Pin HC-SR04	Conexión a ESP32
VCC	VIN / 5V
GND	GND
TRIG	GPIO 5
ECHO	GPIO 18 mediante divisor de voltaje

Precaución eléctrica

El pin ECHO del HC-SR04 puede entregar 5 V. Como la ESP32 opera con lógica de 3.3 V, se recomienda reducir el voltaje de entrada con un divisor antes de conectar al GPIO18.

9.2.1 Esquema del divisor de voltaje

```

ECHO HC-SR04
|
220 ohmios
|
+-----> GPIO18 ESP32
|
220 ohmios
|
GND

```


9.3 Conexión de actuadores

Actuador	Conexión
LED verde	GPIO26 -> resistencia 220 ohmios -> ánodo; cátodo -> GND.
LED rojo	GPIO27 -> resistencia 220 ohmios -> ánodo; cátodo -> GND.
Buzzer activo	GPIO25 -> positivo del buzzer; negativo -> GND.
Servo SG90	Señal amarilla -> GPIO14; rojo -> 5V; negro/café -> GND.

Si el servo se alimenta con una fuente externa, la tierra de esa fuente debe conectarse con el GND de la ESP32 para que la señal de control tenga una referencia común.

10. Funcionamiento e implementación

10.1 Lógica de decisión

El criterio principal del sistema consiste en comparar la distancia medida contra un umbral de 10 cm. Esta regla permite clasificar el cajón sin requerir procesamiento complejo.

```
Si distancia < 10 cm -> estado = OCUPADO
Si distancia >= 10 cm -> estado = LIBRE
```

10.2 Estado libre

- El LED verde se enciende y el LED rojo se apaga.
- El buzzer permanece apagado.
- El servo abre la barrera.
- La ESP32 publica el estado LIBRE y la distancia medida.
- Node-RED actualiza el dashboard con el estado disponible.

10.3 Estado ocupado

- El LED verde se apaga y el LED rojo se enciende.
- El buzzer emite señales intermitentes; la frecuencia aumenta al acercarse el vehículo.
- El servo cierra la barrera.
- La ESP32 publica el estado OCUPADO y la distancia medida.
- Node-RED visualiza el cambio en tiempo real.

10.4 Configuración del firmware

Antes de cargar el firmware se deben configurar las credenciales WiFi y la IP del broker MQTT. La IP corresponde a la computadora donde se ejecuta Mosquitto.

```
const char* ssid = "WIFI";
const char* password = "PASSWORD";
const char* mqtt_server = "192.168.1.65";
```

11. Comunicación MQTT y dashboard

11.1 Tópicos definidos

Tópico	Dato publicado	Ejemplo
escom/iot/equipo1/estacionamiento/distancia	Distancia medida por el HC-SR04 en centímetros.	16.25
escom/iot/equipo1/estacionamiento/estado	Estado lógico del cajón.	LIBRE u OCUPADO

11.2 Calidad de Servicio

Dato	QoS recomendado	Justificación
Distancia	QoS 0	Se publica con alta frecuencia; si se pierde un valor, el siguiente actualiza la medición.
Estado LIBRE/OCUPADO	QoS 1	Es información más importante para la lógica del sistema; conviene confirmar al menos una entrega.

En la implementación básica con PubSubClient se emplea publicación simple, equivalente a QoS 0. En una versión extendida, el tópico de estado puede configurarse desde clientes compatibles para operar con QoS 1.

11.3 Dashboard en Node-RED

El flujo de Node-RED contiene nodos mqtt in para suscribirse al estado y a la distancia. La interfaz usa text para mostrar el estado, gauge para representar la distancia, chart para observar el historial y text adicional para mostrar el valor numérico. La configuración del broker en Node-RED se realiza con servidor localhost y puerto 1883 cuando Mosquitto se ejecuta en la misma computadora.

12. Plan de pruebas y validación

La validación del sistema debe confirmar que cada bloque funcione de forma individual y que la integración completa responda correctamente ante cambios de distancia. El plan de pruebas propuesto cubre conectividad, lectura del sensor, publicación MQTT, visualización en Node-RED y actuación física.

Prueba	Procedimiento	Resultado esperado
Conexión WiFi	Encender la ESP32 y revisar Serial Monitor.	Mensaje de WiFi conectado.
Conexión MQTT	Iniciar Mosquitto y verificar conexión desde ESP32.	Mensaje de MQTT conectado.
Lectura de distancia	Acercar y alejar un objeto del HC-SR04.	Valores de distancia cambian de forma coherente.
Cambio de estado	Colocar objeto a menos de 10 cm y retirarlo.	Cambio entre OCUPADO y LIBRE.

Actuadores	Observar LEDs, buzzer y servo en ambos estados.	LED, buzzer y barrera responden al estado.
Suscripción MQTT	Usar mosquitto_sub en CMD para ambos tópicos.	Se reciben distancia y estado en tiempo real.
Dashboard	Abrir http://localhost:1880/ui .	Se visualiza estado, gauge e historial.

12.1 Comandos de verificación

```
mosquitto_sub -h localhost -t escom/iot/equipo1/estacionamiento/estado
mosquitto_sub -h localhost -t escom/iot/equipo1/estacionamiento/distancia
```

Al modificar la distancia frente al sensor, deben observarse cambios como LIBRE, OCUPADO, LIBRE y valores numéricos de distancia semejantes a 16.25, 7.40 y 15.90 cm.

13. Resultados

El prototipo logró monitorear en tiempo real el estado de un cajón de estacionamiento. La ESP32 publicó la distancia medida por el sensor HC-SR04 y el estado calculado del cajón a través de MQTT. Node-RED recibió la información y la mostró en un dashboard, mientras que los actuadores físicos respondieron conforme a la condición detectada.

Variable observada	Comportamiento obtenido
Distancia	Se actualiza continuamente conforme cambia la separación entre el sensor y el objeto.
Estado del cajón	Cambia entre LIBRE y OCUPADO según el umbral de 10 cm.
LED verde	Permanece encendido cuando el cajón está libre.
LED rojo	Permanece encendido cuando el cajón está ocupado.
Buzzer	Se activa de forma intermitente en condición de ocupado y aumenta su frecuencia con mayor cercanía.
Servo SG90	Simula apertura o cierre de una barrera automática.
Dashboard Node-RED	Permite visualizar estado, distancia y comportamiento histórico.

14. Conclusión

El proyecto demuestra que es posible construir un sistema IoT funcional para monitorear la disponibilidad de un cajón de estacionamiento usando componentes de bajo costo y herramientas de software abiertas. La ESP32 permite integrar lectura de sensores, control de actuadores y comunicación WiFi en una sola placa, mientras que MQTT simplifica el intercambio de mensajes entre el nodo físico y la interfaz de visualización.

El uso de Mosquitto como broker local y Node-RED como dashboard facilita la implementación de una arquitectura clara, modular y replicable. La separación por tópicos permite escalar el sistema hacia múltiples cajones, ya que cada espacio podría tener identificadores propios dentro de una jerarquía MQTT.

El umbral de 10 cm proporciona una regla sencilla para diferenciar entre cajón libre y ocupado. Sin embargo, para entornos reales sería necesario calibrar la distancia, considerar ruido de medición, proteger la electrónica, mejorar la alimentación del servo y fortalecer la seguridad del broker.

15. Recomendaciones y trabajo futuro

- Agregar autenticación al broker MQTT y evitar `allow_anonymous true` fuera de un entorno de pruebas.
- Implementar control de acceso por tópico para impedir publicaciones o suscripciones no autorizadas.
- Usar QoS 1 para el estado del cajón cuando se requiera mayor confiabilidad en la entrega del mensaje.
- Calibrar el umbral de distancia de acuerdo con la ubicación física del sensor y el tamaño del cajón.
- Agregar filtrado por promedio móvil o mediana para reducir fluctuaciones en la medición del HC-SR04.
- Escalar la jerarquía MQTT para múltiples cajones, por ejemplo: `escom/iot/equipo1/estacionamiento/cajon01/estado`.
- Integrar almacenamiento de datos para analizar ocupación histórica y horarios de mayor demanda.
- Diseñar una carcasa o montaje físico que proteja la ESP32, el sensor y el cableado.

Observación para entrega

Los espacios de portada fueron reservados para insertar los escudos de IPN y ESCOM sin alterar el formato del documento.

16. Referencias

- [1] D. E. De la Rosa Gamiño, M. I. López Ramos y M. Morán Vaquero, "Sistema de Estacionamiento Inteligente IoT mediante MQTT", repositorio GitHub estacionamiento-IoT-ESP32, 2026.
<https://github.com/diegoemdg/estacionamiento-IoT-ESP32/blob/main/README.md>
- [2] MQTT.org, "MQTT: The Standard for IoT Messaging". <https://mqtt.org/>
- [3] OASIS Open, "MQTT Version 5.0 OASIS Standard". <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [4] Eclipse Mosquitto, "Eclipse Mosquitto: An open source MQTT broker". <https://mosquitto.org/>
- [5] Node-RED, "Low-code programming for event-driven applications". <https://nodered.org/>
- [6] Espressif Systems, "Arduino Core for ESP32 Documentation".
<https://docs.espressif.com/projects/arduino-esp32/>

17. Anexo: secuencia resumida de ejecución

La siguiente secuencia resume los pasos necesarios para ejecutar el sistema en un entorno local de pruebas.

Paso	Acción
1	Instalar Arduino IDE, soporte ESP32, PubSubClient y ESP32Servo.
2	Instalar Mosquitto y habilitar listener 1883 para pruebas locales.
3	Instalar Node.js, Node-RED y node-red-dashboard.
4	Configurar SSID, contraseña WiFi e IP del broker en el firmware.
5	Cargar el firmware en la ESP32 con la placa ESP32 Dev Module seleccionada.
6	Ejecutar Mosquitto y Node-RED en la computadora.
7	Abrir el dashboard y verificar distancia, estado y respuesta de actuadores.